

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java CompareDomains
Error: java.net.UnknownHostException: No such host is known (helios.ibiblio.org)

C:\Users\user\Desktop\pncw\BCA\6th sem\java>
```

Conclusion

Observation

```
Interface: lo
  Address: 127.0.0.1
  Address: 0:0:0:0:0:0:0:1

Interface: net0

Interface: net1

Interface: ppp0

Interface: net2

Interface: net3

Interface: eth0

Interface: ppp1

Interface: eth1
  Address: fe80:0:0:0:d177:ff2e:b838:c6a3%eth1

Interface: eth2

Interface: wlan0
  Address: 192.168.1.107
  Address: 2400:1a00:5b29:7e5c:d603:7336:bece:2358
  Address: 2400:1a00:5b29:7e5c:549c:459a:5b66:ed65
  Address: fe80:0:0:0:26b2:9245:92b2:5cd4%wlan0

Interface: wlan1

Interface: wlan2

Interface: eth3

Interface: net4

Interface: wlan3
  Address: fe80:0:0:0:66ec:3fe8:e09:3b20%wlan3
```

Conclusion

```

Name: lo
Display Name: Software Loopback Interface 1
Is Up?: true
Is Loopback?: true
Is Virtual?: false
Supports Multicast?: true
MTU: -1
-----
Name: net0
Display Name: Microsoft 6to4 Adapter
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
-----
Name: net1
Display Name: WAN Miniport (PPTP)
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
-----
Name: ppp0
Display Name: RAS Async Adapter
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
-----
Name: net2
Display Name: Microsoft IP-HTTPS Platform Adapter
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
-----
Name: net3
Display Name: WAN Miniport (L2TP)
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
-----

```

```

Name: eth0
Display Name: Microsoft Kernel Debug Network Adapter
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
-----
Name: ppp1
Display Name: WAN Miniport (PPPOE)
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: -1
-----
Name: eth1
Display Name: Bluetooth Device (Personal Area Network) #2
Is Up?: false
Is Loopback?: false
Is Virtual?: false
Supports Multicast?: true
MTU: 1500
-----

```

Conclusion

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java CheckReachability  
Pinging: www.google.com  
Host is reachable.
```

Conclusion

The program successfully demonstrated how to check if a remote host is reachable using Java's `isReachable()` method. This is useful for basic network diagnostics and testing connectivity between systems.

Observation

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java SpamCheck
Enter a message:
Click here to win free macbook.
This message is SPAM.
```

Conclusion

The program successfully demonstrated a basic spam detection mechanism using keyword matching. It shows how simple text analysis can be used to identify suspicious or unwanted messages.

Observation

```
Processing Web Server Log File:
IP Address: 192.168.1.5
Method: GET
Resource: /index.html
Status Code: 200
-----
IP Address: 192.168.1.6
Method: POST
Resource: /login
Status Code: 302
-----
IP Address: 192.168.1.7
Method: GET
Resource: /about.html
Status Code: 200
-----
```

Conclusion

Java's file-handling features make it easy to process server log files. The program demonstrated how to read log entries, split them into components, and analyze useful information for monitoring server performance.

Observation

```
Full URL: https://www.example.com:8080/path/to/resource?query=test
Protocol: https
Host: www.example.com
Port: 8080
Path: /path/to/resource
Query: query=test
File: /path/to/resource?query=test
```

Conclusion

By using the URL class and its getter methods, we can easily extract individual parts of a URL. This demonstrates how Java provides built-in functionality for URL parsing without manually splitting strings.

```
C:\Users\user\Desktop\pncw\BCA\6th  
http protocol is SUPPORTED.  
https protocol is SUPPORTED.  
ftp protocol is SUPPORTED.  
file protocol is SUPPORTED.  
mailto protocol is SUPPORTED.  
jar protocol is NOT supported.
```

Conclusion

By using the URL class and handling MalformedURLException, we can check which protocols the virtual machine supports. This method provides a simple way to test network protocol handling in Java.

Observation

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>javac DownloadWebPage.java  
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java DownloadWebPage  
Web page downloaded successfully!
```

Conclusion

Using Java's URL and URLConnection classes, we can download a web page's content and save it locally. This demonstrates basic web scraping and network programming in Java.

```
Base URI: https://www.example.com/folder/  
Relative URI: page.html  
Resolved URI: https://www.example.com/folder/page.html
```

Conclusion

The program demonstrates how Java's URI class can resolve relative URIs to absolute URIs using the `resolve()` method, which is helpful in web development and resource management.

Observation

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>javac DownloadObject.java  
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java DownloadObject  
File downloaded successfully!
```

Conclusion

By using Java's URL and openStream() methods along with FileOutputStream, we can download objects or files from the internet efficiently. This demonstrates basic file handling and network programming in Java.

```
C:\Users\user\Desktop\pncw\BCA\6th sem\java>java XwwwFormEncoded
x-www-form-urlencoded string:
name=Deep&email=deep.drsn%40example.com&message=Hello%2C+how+are+you%3F
```

Conclusion

The program demonstrates how Java's URLEncoder class converts strings into x-www-form-urlencoded format. This is essential for sending safe and properly encoded form data over HTTP.

Observation

```
Server response:
{
  "args": {
    "age": "20",
    "name": "Deep"
  },
  "headers": {
    "Accept": "text/html, image/gif, image/jpeg, *; q=.2, */*; q=.2",
    "Host": "httpbin.org",
    "User-Agent": "Java/17.0.10",
    "X-Amzn-Trace-Id": "Root=1-6938089e-298a94b54e23f80f028bc6fd"
  },
  "origin": "27.34.73.100",
  "url": "https://httpbin.org/get?name=Deep&age=20"
}
```

Conclusion

By using Java's URL and HttpURLConnection classes, we can communicate with server-side programs using HTTP GET. This is useful for fetching data or interacting with web services.